

Furmosa HQ 系統擴充計畫

店家 POS + 顧客預約美容系統

版本：v1.0 制定日期：2026-07-21

制定人：C Ming（產品 / 唯一開發者）+ Cursor AI Agent

0. 文件目的

本文件記錄：

1. 目前系統的現況與效能診斷（有實際程式碼證據，非猜測）
2. 三個確認需求（店家帳號模型、一鍵叫貨、顧客預約系統）的規格整理
3. 分階段（Phase 0 ~ 4）的落地路線圖，每階段都能獨立上線、獨立驗收
4. 技術決策理由，以及開發前仍需要你確認的細節

核心原則： 目前只有 1 位開發者 + AI Agent，合作店家規模是 7 家（一年內 9 家）。這代表**不需要、也不應該**現在就做「大公司等級」的微服務拆分或重寫，而是用**同一套 Next.js 專案 + 漸進式強化**的方式，把風險與維運複雜度壓到最低。

1. 現況總覽

項目	現況
技術棧	Next.js 14 App Router、TypeScript、Prisma ORM、Supabase Postgres、Vercel 部署
資料模型	32 個 Prisma model，核心涵蓋訂單、庫存、寄賣店家（Merchant）、會員、換罐計畫、訂閱
使用者	目前僅總公司內部同仁登入使用（User 模型），無店家 / 顧客對外帳號
合作店家	7 家（Merchant 資料表），一年內預計 +2 家 → 共 9 家
團隊	1 人（產品 / 決策）+ AI Agent 開發，無其他工程師

2. 現況效能診斷（有程式碼依據）

2.1 Dashboard 頁面近 20 次資料庫往返，且完全序列執行

```
features/dashboard/queries.ts
runInBatches(tasks, batchSize = 1) // 16 個查詢逐一排隊，無並行
+ 3 個查詢完全在批次機制外，額外再序列跑一次
= 每次打開 Dashboard 約 19 趟資料庫來回
```

註解原文：「Supabase pooler 連線數有限；Vercel 上 batchSize=1 最穩」— 這是**用犧牲速度換穩定性**的暫時解法，不是長期架構。

2.2 完全沒有快取層

Dashboard、店家列表、商品列表等頁面，每次都是**即時重新計算**，沒有 ISR、沒有 Redis / Vercel Runtime Cache。這是目前「感覺很慢」的最大原因，也是跟「大公司網站速度」差距最大的地方。

2.3 高頻查詢欄位缺索引

`Order.merchantId`、`Order.customerId` 完全沒有索引，但：

- Dashboard 用 `groupBy(['merchantId'])` 統計寄賣店排行
- 店家詳情頁、客戶詳情頁都要用 `merchantId / customerId` 撈該店家 / 客戶的訂單

資料量還小時感覺不出來，但店家一多、訂單一多，這兩個欄位會是第一個爆掉的查詢。

2.4 單一連線池，所有功能共用

32 個 model 全部擠在一個 Supabase Postgres + PgBouncer 連線池。未來 POS + 預約系統上線後，尖峰時段（例如週末很多店同時操作 POS）會跟後台互搶連線，放大現有問題。

結論：現在的慢，是「單體應用 + 零快取 + 序列查詢 + 缺索引」的結果，是工程實踐差距，不是硬體不夠。這些都可以在不重寫系統的前提下逐步解決（見 Phase 0）。

3. 確認需求規格

3.1 店家帳號模型

一間店 = 一個 Merchant = 一組帳號

- 沿用現有 Merchant 資料表作為「店家」主體，不用新建一套獨立系統
- 每個 Merchant 對應一組登入帳號（先做「一店一號」，同店員工共用；未來若要細分店長 / 店員權限可再擴充）
- 店家登入後只能看到、操作自己店的資料（叫貨、庫存、預約），看不到其他店或總公司後台

3.2 一鍵叫貨（補貨申請）

項目	規格
選品方式	二選一：① 店家自己挑要叫的產品與數量 ② 交給總公司決定（店家只送出「我要叫貨」的申請）
產品範圍	限定在「換罐計畫」系統裡已有的產品種類，不是全商品目錄
送出後流程	店家送出申請 → 進入「待審核」狀態 → 總公司審核（可調整品項 / 數量）→ 審核通過後回填「預計到貨日期」→ 店家看到到貨日期與狀態
對應現有機制	這個流程本質上是現有「寄賣進貨出貨單」（Shipment type = merchant_restock）的前端申請入口 + 審核關卡，資料骨幹可以重用，不必另建一套倉儲邏輯

3.3 顧客預約美容系統

項目	規格
預約動線	顧客先選店家 → 系統顯示該店有空的時段 → 可選擇指名技師
技師異動	若預約後該技師臨時沒空，店家要主動、提前通知客人改期或改人（不是系統自動處理，是給店家一個「這筆預約技師有異動，請聯絡客人」的提示 / 工具）
訂金	不需要
客人評分	需要「評分客人」機制（店家對到店顧客評分 / 標註，可能用於信譽、防止不良客人等）

通知	需整合 LINE 通知（預約成立、店家異動通知、提醒等），系統已有 LINE Messaging API 串接基礎可延伸
----	--

開發前仍需要你確認的細節（不影響現在開始 Phase 0，但會影響 Phase 4 設計）：

1. 「評分客人」的呈現方式：星等（1-5）？標籤（如：準時／爽約／友善）？評分是否要跨店共享（A 店標記的顧客，B 店看不看到）？
2. 技師班表怎麼輸入：店家自己在後台排班，還是先用「每天固定時段」簡化版？
3. 技師異動通知：只是「站內提示 + 店家自己用 LINE/電話聯絡」，還是要系統自動發 LINE 訊息給客人？
4. 預約可否線上取消／改期，還是都要店家端操作？

4. 系統架構總覽（目標樣貌）

維持同一個 Next.js 專案，用路由群組（route group）區分三種使用情境，共用同一個資料庫與帳務核心：

```

app/
├─ (main)/      ← 現有：總公司後台（既有功能不變）
├─ (pos)/       ← 新增：店家 POS（叫貨、庫存、預約管理）
└─ (booking)/   ← 新增：顧客預約前台（免登入或輕量驗證）

```

為什麼不拆成微服務／獨立系統

考量	說明
團隊規模	1 人 + AI，維運多套系統的成本會直接拖垮開發速度
規模	9 家店的量級，單一 Postgres + 合理快取完全撐得住，微服務帶來的效益遠小於複雜度成本
資料一致性	POS 庫存、叫貨、訂單、結算本質上是同一套帳，拆開來會需要額外處理跨系統同步，得不償失

「像 Google 一樣快」的現實做法：不是換更貴的伺服器，而是補齊 (1) 快取層 (2) 資料庫索引 (3) 非同步/背景處理 (4) 靜態化公開頁面（顧客預約前台這種匿名流量頁面最適合）。Vercel 本身已提供全球 CDN / 邊緣網路，這部分不需要自己重造。

5. 資料模型設計草案（Phase 1 ~ 4 會逐步新增）

這裡只列草案方向，正式欄位會在對應 Phase 開工時，依你確認的細節（見 3.3 待確認清單）定案。

```

MerchantUser      店家登入帳號（email/手機 + 密碼 + merchantId + role）

RestockRequest     叫貨申請單
  merchantId, mode(self_select|company_decide), status,
  requestedAt, reviewedAt, confirmedArrivalDate, note
RestockRequestItem 申請品項（限換罐計畫產品範圍）
  requestId, productId, quantity

Technician        店家技師
  merchantId, name, active
TechnicianSlot     技師可預約時段 / 班表

```

```
    technicianId, date, startTime, endTime, isBooked
Appointment          預約
    merchantId, customerId, technicianId, slotId,
    status(confirmed|changed|cancelled|completed|no_show), note
CustomerRating       店家對顧客的評分
    merchantId, customerId, rating, tag, note
```

沿用不變的既有基礎：Merchant、MerchantStock / MerchantStockTxn（店家庫存流水）、Shipment（出貨/進貨單）、LINE 訊息推播架構（lib/line/*）、Web Push（lib/web-push.ts）。

6. 分階段路線圖

每個 Phase 都設「完成定義（DoD）」，確保每階段做完就是「真的做完、可驗收」，不是模糊的進度。

Phase 0：效能止血

[本次立即開始](#)

不改變任何架構 / 資料模型。

任務	內容
補齊資料庫索引	Order.merchantId、Order.customerId 加索引
Dashboard 查詢快取化	用 Next.js 內建快取機制，KPI 數字改成短時間（如 30 ~ 60 秒）快取，不必每次即時全算
調整查詢併發策略	修正目前不一致的序列/併發邏輯，把漏在批次機制外的查詢一併納管，並在連線池可承受範圍內提高並行度
連線池設定文件更新	更新 .env.example / DEPLOY.md 的建議設定，反映快取後的實際連線壓力

DoD: Dashboard 頁面資料庫往返次數明顯下降、高頻查詢有索引覆蓋、現有單元測試與 build 全部通過、不影響任何現有功能行為。

Phase 1：多租戶帳號模型

- 新增 MerchantUser（店家登入帳號）與對應登入頁、session 機制（沿用現有 bcryptjs + jose JWT 作法）
- 建立「店家只能看自己資料」的權限中介層

DoD: 店家可用自己帳密登入，看到的資料範圍僅限自己的 Merchant。

Phase 2：一鍵叫貨（Restock Request）

- 依 3.2 規格建 RestockRequest / RestockRequestItem
- 串接既有 merchant_restock 出貨流程，總公司審核後自動產生對應進貨單
- 通知：審核結果與到貨日期回填給店家（站內 + 可選 LINE）

DoD: 店家在 POS 送出叫貨申請 → 總公司後台看到待審核清單並可調整、確認到貨日 → 店家看到狀態更新。

Phase 3：店家 POS（銷售 / 庫存記錄）

- 延伸現有 MerchantStock / MerchantStockTxn，做成店家自助操作的簡化介面（記錄銷售、盤點）

DoD: 店家可在自己帳號下直接記錄銷售與庫存異動，總公司後台即時看到。

Phase 4: 顧客預約美容系統

- 技師與班表管理（店家端）
- 顧客預約前台：選店 → 選時段 → 選技師
- 技師異動的店家端提示工具
- 評分客人機制
- LINE 通知串接（預約成立、異動提醒）

DoD: 顧客可完成線上預約並收到 LINE 通知；店家可管理班表、標記技師異動、對顧客評分。

Phase 5: 效能與基礎設施持續強化 貫穿全程

- 導入正式快取層（依流量觀察決定是否需要 Redis/Upstash）
- 重運算（月結算、報表）改背景處理
- 基本錯誤追蹤與慢查詢監控

7. 風險與應對

風險	應對
單人開發，變更範圍大	每個 Phase 拆成小任務、獨立 PR、每次改完都跑既有測試，不做「一次性大改」
資料庫連線池在多系統並存後吃緊	Phase 0 先解決現有瓶頸；後續每個 Phase 上線前重新評估連線壓力
LINE 訊息額度 / 限流	沿用現有節流機制（lib/line/*-throttle.ts），新增通知場景時一併考慮頻率限制
需求細節未定（評分機制、班表輸入方式等）	列於 3.3，建議在 Phase 4 開工前一次性確認，不影響 Phase 0 ~ 3 進度

8. 下一步

1. 本文件確認後，Phase 0 立即開始（見下方執行紀錄）
2. Phase 1 開工前，需要你確認：店家帳號要不要區分「店長 / 店員」角色，還是先用一店一號即可
3. Phase 4 開工前，需要你確認 3.3 列出的 4 個問題

9. Phase 0 執行紀錄

狀態：已完成並通過驗證（2026-07-21）

變更	檔案	說明
補齊索引	prisma/schema.prisma、 prisma/migrations/20260721130000_order_merchant_customer_index/migration.sql	新增 Order.merchantId、

		Order.customerId 索引
Dashboard 快取化	features/dashboard/queries.ts	用 unstable_cache 包住 loadDashboardData(), 30 秒內重複請求不再打資料庫；同一份資料所有使用者共享同一份快取
併發策略 調整	features/dashboard/queries.ts	runInBatches 批次大小由 1 提高到 3；原本漏在批次機制外、完全序列執行的「寄賣店排行 / 本月下單會員 / 回購率」三個查詢，改為兩個互不相依的查詢並行執行，且全部套用既有的 withDbRetry 重試機制
連線池設定文件	.env.example、DEPLOY.md	更新註解說明快取後的連線壓力模型，建議 connection_limit>=10

已知取捨：Dashboard KPI 現在允許最多 30 秒的資料延遲（例如新增訂單後，Dashboard 數字最慢 30 秒後才會更新）。任務（Task）異動仍會呼叫 revalidatePath('/dashboard')，但這只會刷新頁面本身，KPI 數字仍依快取時間為準。若之後覺得 30 秒延遲不能接受，可改用 revalidateTag 搭配在各筆訂單/任務異動處補上標籤失效呼叫（成本較高，非 Phase 0 範圍）。

驗證

- npx tsc --noEmit：通過
- npm test：49 個測試中 48 通過；1 個既有失敗（lib/jar-exchange/_tests_/jar-exchange.test.ts）在改動前即已失敗，與本次變更無關
- npx next build：成功，/dashboard 路由正常編譯

尚待你（在 Supabase 正式環境）執行

- npx prisma migrate deploy（或下次部署時自動套用）以套用新索引 migration
- 觀察 Vercel 上 Dashboard 的實際載入時間變化，決定是否要把快取秒數從 30 秒再往上調